Práctica 1: Programación secuencial. Sesión 1.

Asignatura: Arquitectura de Computadores
Grado en Ingeniería Informática
Universidad de Valencia

Curso 2025–2026

1. Introducción

El modelo de programación basado en el paralelismo de datos, utilizado tradicionalmente en procesadores vectoriales y en arquitecturas SIMD, evolucionó de forma natural hacia el modelo SPMD (*Single Program, Multiple Data*). Este modelo comenzó a utilizarse en los años 80 en sistemas multiprocesador con memoria compartida, junto con los denominados compiladores paralelizantes, desarrollados por distintos fabricantes (IBM, SGI, Cray, entre otros). Este tipo de programación ha sido especialmente beneficioso para científicos e ingenieros que trabajan con problemas de gran tamaño.

En el modelo SPMD, el programador diseña inicialmente el programa de forma secuencial, y es el compilador el encargado de generar la versión paralela del código, principalmente mediante la paralelización de bucles. Además, el programador puede intervenir de forma explícita en este proceso, indicando al compilador qué partes del código deben paralelizarse y cómo deben tratarse las variables. Esto permite desarrollar aplicaciones paralelas de manera relativamente sencilla, aunque para obtener mejoras de rendimiento significativas suele ser necesario analizar en detalle las dependencias de datos del algoritmo o incluso sustituirlo por otro más adecuado para la ejecución paralela.

OpenMP se ha consolidado como uno de los estándares más utilizados para la programación SPMD en sistemas con memoria compartida, como los procesadores multinúcleo actuales. A diferencia de los compiladores paralelizantes automáticos, OpenMP requiere que el programador indique explícitamente cómo debe paralelizarse el código, mediante directivas insertadas en el propio programa, siendo también responsable de garantizar la corrección de los resultados.

Esta práctica consta de tres sesiones de laboratorio. En la primera sesión se revisarán de forma guiada y práctica los conceptos básicos vistos en clase, mientras que en las dos sesiones posteriores se abordará la paralelización del problema de estudio. El objetivo de esta primera sesión es familiarizar al estudiante con las principales herramientas que ofrece OpenMP para el diseño y la ejecución de programas paralelos en sistemas multinúcleo.

Como en el resto de sesiones de laboratorio, se pretende afianzar los conocimientos teóricos y continuar desarrollando las técnicas básicas para la toma de datos y su representación mediante tablas o gráficas. Para ello, antes de la sesión se pedirá a los miembros de cada grupo que lean la documentación relacionada con los fundamentos teóricos y las herramientas que se utilizarán. Esta preparación previa facilitará el uso del entorno de trabajo y permitirá resolver dudas de forma más eficaz.

Durante las sesiones de laboratorio es importante mantener una buena dinámica de trabajo en grupo. Para ello se requiere puntualidad, atención a las indicaciones del profesor y una actitud activa. En esta primera sesión esto es especialmente importante, ya que se explicarán las normas de funcionamiento necesarias para el correcto desarrollo del resto de sesiones.

1.1. Objetivos

El objetivo general de esta práctica es aprender a trabajar con OpenMP y estudiar el impacto de las distintas opciones de compilación en la ejecución de los programas. Asimismo, se pondrá de manifiesto la importancia de conocer la arquitectura del computador para aprovechar adecuadamente sus recursos.

Al finalizar esta práctica, el estudiante será capaz de:

- Compilar y ejecutar programas utilizando OpenMP.
- Utilizar la directiva `#pragma omp parallel` y los constructores de distribución del trabajo `for` y `sections`.
- Emplear las cláusulas sobre variables `private` y `firstprivate`.

- Utilizar funciones de tiempo de ejecución como `omp_get_thread_num`, `omp_get_num_threads` y `omp_set_num_threads`.
- Aplicar distintas políticas de planificación de hilos.
- Usar directivas de acceso a variables compartidas.
- Configurar y utilizar las variables de entorno más relevantes de OpenMP.

1.2. Recursos

Para el desarrollo de las sesiones de laboratorio se utilizará un ordenador personal desde el cual se accederá al servidor `boe.uv.es`, que dispone de sistema operativo **Linux CentOS 6.5**.

El servidor `boe` cuenta con dos procesadores multinúcleo **Intel Xeon E5-2620V2** a **2,1 GHz**, con **6 núcleos cada uno** (un total de **12 CPU con hyper-threading**), fabricados en tecnología de **22 nm**, con una potencia de **80 W** y una memoria principal **DDR3 a 1600 MHz** de **64 GB**. Cada núcleo dispone de memoria caché **L2 de 0,25 MB** y una caché **L3 unificada de 15 MB** (2,5 MB por núcleo).

El servidor `boe` está conectado mediante una red interna **Gigabit Ethernet** a **8 nodos de cálculo**, denominados *compute-0-1* a *compute-0-7*. Cada nodo dispone de un procesador **Intel Xeon E5-2620V2** similar al del servidor principal, pero con **32 GB de memoria principal**.

El compilador de C instalado en el sistema es **GNU gcc versión 9.4.0**, que incluye soporte para **OpenMP 4.5**.

1.3. Aprendizaje y evaluación del laboratorio

El aprendizaje y la evaluación de la práctica comienzan antes de la sesión de laboratorio. Se recomienda que los estudiantes se reúnan previamente con su grupo de trabajo (preferiblemente de forma telemática) para preparar cada sesión. Este trabajo previo consiste en repasar los conceptos teóricos relacionados con la práctica y comenzar a realizar las actividades propuestas, de modo que las dudas puedan resolverse durante la sesión o en tutorías.

La evaluación de esta práctica se realizará de la siguiente forma: la primera sesión se evaluará mediante una prueba individual, mientras que las sesiones 2 y 3 se evaluarán mediante una memoria que recogerá el conjunto del trabajo realizado.

2. Trabajo previo al laboratorio – Pre-Lab

Para poder aprovechar al máximo la sesión de laboratorio es muy importante leer con atención el enunciado de la práctica y realizar previamente parte de las actividades propuestas, dedicando aproximadamente el tiempo indicado.

Con el objetivo de no superar el tiempo máximo estimado, los miembros del grupo de trabajo deberán repartirse las tareas y realizar posteriormente una puesta en común. Una mala organización del trabajo previo, o que una única persona intente realizar todas las actividades, puede provocar que no se completen todas las tareas o que se dedique más tiempo del recomendado, en detrimento de otras actividades del curso.

El trabajo previo permitirá llegar a la sesión de laboratorio con una mejor comprensión de los conceptos y facilitará la resolución de dudas durante la práctica.

3. Trabajo en el laboratorio – In-Lab

Durante esta sesión se aprenderá a utilizar los elementos básicos de OpenMP a partir de un conjunto de programas de ejemplo. En los apartados siguientes se propondrán distintas activida-

des prácticas, acompañadas de cuestiones que deberán resolverse o de acciones que será necesario experimentar directamente sobre el código.

El objetivo de esta sesión es que el estudiante se familiarice con el uso de OpenMP de forma progresiva y guiada. La evaluación de esta sesión será de carácter individual.

3.1. Ejercicio 1: Programa "Hola mundo" con OpenMP

Vamos a comenzar la práctica con un programa sencillo del tipo *Hola mundo*, utilizando la directiva `parallel` de OpenMP. Esta directiva crea un conjunto de hebras (hilos) que se ejecutan de forma paralela.

A continuación se muestra el código del programa:

```c
#include <omp.h>
#include <stdio.h>

int main() {
    int nthreads, thread;

    #pragma omp parallel private(nthreads, thread)
    {
        thread   = omp_get_thread_num();
        nthreads = omp_get_num_threads();
        printf("Hola soy el hilo = %d de %d\n", thread, nthreads);
    }

    return 0;
}
```

Como puede observarse, el código comienza incluyendo el fichero de cabecera `<omp.h>`, necesario para poder utilizar las funciones y directivas de OpenMP.

La directiva `parallel` afecta al bloque de código delimitado por llaves. La cláusula `private` indica que cada hebra tendrá su propia copia local de las variables `nthreads` y `thread`, evitando interferencias entre hilos.

Para compilar el programa se utilizará la siguiente instrucción:

```
gcc -fopenmp hola.c -o hola
```

Y para ejecutarlo:

```
./hola
```

La salida del programa dependerá del número de hebras que se ejecuten. Por defecto, OpenMP lanza tantas hebras como núcleos tenga el computador en el que se está ejecutando el programa.

A continuación, realiza las siguientes pruebas y responde a las cuestiones planteadas:

1. Cambia el número de hebras que se ejecutan en paralelo utilizando la función `omp_set_num_threads`. Prueba con un número de hebras mayor y menor que el número de núcleos del computador.

2. ¿Qué ocurre si se elimina la cláusula `private` de la directiva `parallel`? Describe el comportamiento observado.

3. Crea una variable de tipo `int`, asígnale el valor 100 y pásala a la región paralela como `private`. Comprueba el valor que toma dicha variable en cada hebra. Repite el experimento utilizando la cláusula `firstprivate` y compara los resultados.

4. Modifica el código de la región paralela para que únicamente la hebra con identificador 0 imprima por pantalla el valor de la variable de tipo `int` del apartado anterior.

3.2. Ejercicio 2: Bucles paralelos con `parallel` y `for`

En este ejercicio combinaremos la directiva `#pragma omp parallel` con `#pragma omp for` para paralelizar un bucle. Esta combinación permite repartir las iteraciones del bucle entre las hebras creadas, de manera que cada hilo ejecuta una parte del trabajo.

A continuación se muestra el código base del ejercicio:

```c
#include <omp.h>
#include <stdio.h>
#include <unistd.h>

#define N 10
#define NTHREADS 4

int main() {
    int tid, i;

    omp_set_num_threads(NTHREADS);

    #pragma omp parallel private(tid)
    {
        tid = omp_get_thread_num();

        #pragma omp for
        for (i = 0; i < N; i++) {
            sleep(i);
            printf("El hilo %d ejecuta la iteracion %d\n", tid, i);
        }
    }

    return 0;
}
```

Para compilar el programa se utilizará el mismo comando que en el ejercicio anterior:

```
gcc -fopenmp bucles.c -o bucles
```

Y para ejecutarlo:

```
./bucles
```

Utilizando este código como base, realiza las siguientes pruebas:

1. Compila y ejecuta el programa. Comprueba:
   - cuántas iteraciones ejecuta cada hebra,
   - si las iteraciones asignadas a un mismo hilo son consecutivas o no,
   - y si la distribución de la carga está equilibrada.
2. Prueba distintos tipos de planificación con la cláusula `schedule({static|dynamic|guided|auto})` de la directiva `#pragma omp for`, intentando obtener en cada caso la mejor distribución del trabajo. Describe el comportamiento observado en cada caso.

   Por ejemplo:

   ```c
   #pragma omp for schedule(static)
   ```

   ```c
   #pragma omp for schedule(dynamic)
   ```

   ```c
   #pragma omp for schedule(guided)
   ```

   ```c
   #pragma omp for schedule(auto)
   ```

3.3. Ejercicio 3: Cálculo de π y variables compartidas

Después de practicar directivas y cláusulas para crear hebras, repartir trabajo y planificar iteraciones, en este ejercicio nos centraremos en un aspecto muy importante: el uso de **variables compartidas** y los problemas que aparecen cuando varios hilos acceden a la misma variable al mismo tiempo.

Como ejemplo, utilizaremos un programa que aproxima el valor de π mediante integración numérica. El siguiente código reparte las iteraciones del bucle entre las hebras y acumula en la variable `sum` el resultado parcial de cada iteración:

```c
#include <math.h>
#include <stdio.h>
#include <omp.h>

int main(int argc, char *argv[]) {
    int n, i;

    printf("Introduce la precision del calculo (n > 0): ");
    scanf("%d", &n);

    double PI25D = 3.141592653589793238462643;
    double h = 1.0 / (double) n;
    double sum = 0.0;

    #pragma omp parallel for shared(sum)
    for (i = 1; i <= n; i++) {
        double x = h * ((double)i - 0.5);
        sum += (4.0 / (1.0 + x * x));
    }

    double pi = sum * h;

    printf("El valor aproximado de PI es: %f, con un error de %f\n",
           pi, fabs(pi - PI25D));

    return 0;
}
```

En este programa, todas las hebras comparten la variable `sum`. Esto puede provocar un **problema de condición de carrera** (*race condition*), ya que varios hilos pueden intentar actualizar `sum` simultáneamente, causando resultados incorrectos.

Realiza las siguientes actividades:

1. Ejecuta el programa con precisiones $n = 10$, 100 y 1000. Comprueba los valores obtenidos y justifica el resultado observado. (Pista: ¿por qué a veces el error puede ser mayor de lo esperado, incluso aumentando n?)

2. Para corregir el problema, es necesario asegurar la **atomicidad** del acceso a la variable `sum`. Existen tres formas típicas de hacerlo en OpenMP:

   a) `#pragma omp critical`

   b) `#pragma omp atomic`

   c) la cláusula `reduction`

   Modifica el código para que funcione correctamente utilizando cada una de las tres soluciones.

A continuación se muestran plantillas mínimas de cada opción:

a) Usando `critical`:

```
#pragma omp parallel for shared(sum)
for (i = 1; i <= n; i++) {
    double x = h * ((double)i - 0.5);
    double temp = (4.0 / (1.0 + x * x));

    #pragma omp critical
    {
        sum += temp;
    }
}
```

b) Usando `atomic`:

```
#pragma omp parallel for shared(sum)
for (i = 1; i <= n; i++) {
    double x = h * ((double)i - 0.5);
    double temp = (4.0 / (1.0 + x * x));

    #pragma omp atomic
    sum += temp;
}
```

c) Usando `reduction`:

```
#pragma omp parallel for reduction(+:sum)
for (i = 1; i <= n; i++) {
    double x = h * ((double)i - 0.5);
    sum += (4.0 / (1.0 + x * x));
}
```

3.4. Ejercicio 4: `sections`

En este ejercicio veremos cómo un programa con dos tareas independientes, que inicialmente se ejecutan de forma secuencial, puede ejecutarse más rápido si ambas tareas se ejecutan en paralelo mediante la directiva `#pragma omp sections`.

A continuación se presenta un programa de ejemplo en el que se ejecutan dos tareas de manera secuencial y se mide el tiempo total de ejecución:

```
#include <omp.h>
#include <stdio.h>
#include <unistd.h>

void tarea_uno() {
    sleep(2);  // retardo de 2 segundos
}

void tarea_dos() {
    sleep(4);  // retardo de 4 segundos
}

int main() {
    double timeIni, timeFin;
    int tid;

    #pragma omp parallel private(timeIni, timeFin, tid)
    {
```

```
        tid = omp_get_thread_num();

        timeIni = omp_get_wtime();

        printf("(hilo:%d) Ejecutando tarea 1\n", tid);
        tarea_uno();

        printf("(hilo:%d) Ejecutando tarea 2\n", tid);
        tarea_dos();

        timeFin = omp_get_wtime();

        printf("(hilo:%d) Tiempo requerido = %f segundos\n",
               tid, timeFin - timeIni);
    }

    return 0;
}
```

A partir de este código, realiza las siguientes actividades:

1. Compila y ejecuta el programa con **2 hilos**. Comprueba que el tiempo total obtenido es congruente con la duración de las tareas.
2. Modifica el programa para utilizar `#pragma omp sections` y ejecutar las dos tareas en paralelo. Haz que se muestre qué hilo ejecuta cada `section`.
3. Establece una medida de tiempo justo al salir del bloque `sections`. Si una tarea tarda más que la otra, podría esperarse que cada hebra registre tiempos distintos (con una diferencia aproximada a la diferencia de duración entre tareas). Comprueba si esta suposición se cumple. Si no se cumple, piensa qué habría que hacer para que ocurra. **Ayuda:** revisa la cláusula `nowait`.
4. Aumenta el número de hilos y observa qué ocurre cuando hay **más hebras que secciones**. Describe el comportamiento.
5. Realiza el caso contrario: provoca que haya **más secciones que hilos** (por ejemplo, disminuyendo el número de hilos o añadiendo más tareas). Observa qué ocurre en el ejemplo y descríbelo.

3.5. Ejercicio 5: Coste de creación de hilos y selección de núcleos

La creación y destrucción de hilos (*fork + join*) introduce un sobrecoste en la ejecución de programas paralelos con OpenMP. Por ello, es importante estimar cuánto cuesta crear hilos (según el número de hebras) y decidir cuándo *merece la pena* paralelizar una región de código para un tamaño de problema determinado.

Como base, partimos del siguiente programa secuencial, que mide el tiempo de ejecución del algoritmo en versión secuencial y deja un hueco para implementar la versión paralela:

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

int main() {
    unsigned int N;
    int i, *x, *y;

```

```
    printf("Introduce el tamano del vector: ");
    scanf("%u", &N);

    double timeIni, timeFin;

    x = (int*) malloc(N * sizeof(int));
    y = (int*) malloc(N * sizeof(int));

    timeIni = omp_get_wtime();

    // Secuencial
    for (i = 1; i < (int)N; i++) {
        x[i] = y[i-1] * 2;
        y[i] = y[i] + i;
    }

    timeFin = omp_get_wtime();
    printf("Tiempo secuencial = %f milisegundos\n", (timeFin - timeIni) * 1000);

    timeIni = omp_get_wtime();

    // Paralelo
    // Escribe aqui el mismo algoritmo paralelizado

    timeFin = omp_get_wtime();
    printf("Tiempo paralelo   = %f milisegundos\n", (timeFin - timeIni) * 1000);

    free(x);
    free(y);

    return 0;
}
```

A partir de este código, realiza las siguientes tareas:

1. Analiza las dependencias de datos del algoritmo secuencial. Explica por qué, tal y como está escrito, no se puede paralelizar directamente y qué cambios son necesarios para hacerlo paralelizable.

2. Implementa la versión paralela del algoritmo en el espacio reservado del código anterior.

3. Ejecuta el código con $N = 10000$ variando el número de hilos (`1, 2, 4, 8`) utilizando **un único núcleo**. Para ello:

   - fija la variable de entorno `GOMP_CPU_AFFINITY=0` para que todos los hilos se ejecuten en el *core 0*,
   - y evita la migración de hilos con `OMP_PROC_BIND=TRUE`.

   A partir de los tiempos, infiere cuál sería el coste aproximado de creación/destrucción de hilos (fork+join).

4. Ejecuta varias veces el código para $N = 2000$, 20000, 200000 y 2000000. Observa el tiempo obtenido al ejecutarlo con **2 hilos** y **2 núcleos**.

5. A partir de los datos del punto anterior, utiliza la cláusula `if` en la directiva de OpenMP para que la región se ejecute en paralelo **solo a partir de un valor de** N que garantice una mayor eficiencia.

6. Utiliza la variable de entorno `GOMP_CPU_AFFINITY` para ejecutar el código con **2 hilos** y valores elevados de N:

   a) Utilizando 2 núcleos de la misma CPU.

   b) Utilizando 2 núcleos de CPUs distintas.

   Comprueba los tiempos obtenidos y justifica las diferencias observadas.

4. Evaluación de la práctica

La evaluación de esta sesión se realizará al inicio de la siguiente sesión de laboratorio mediante un examen individual. El examen constará de preguntas de tipo test y de respuesta corta, y tendrá una duración máxima de **30 minutos**.